

UFU/FACOM/BCC

Disciplina: Algoritmos e Estruturas de Dados II

Ref: Primeira Prova Valor: 40 Data: 16/12/2025

Nome:

QUESTÕES:

1 – (valor: 2,0) O algoritmo Selection Sort realiza quantas trocas no pior caso para um vetor de tamanho n ?

- A) $O(n^2)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n/2)$

2 – (valor: 4,0) Classifique como V(Verdadeiro) ou F(Falso):

- 1. () O Bubble Sort pode parar mais cedo se, em uma passada, nenhuma troca for realizada.
- 2. () O Merge Sort possui caso médio $O(n^2)$.
- 3. () O Insertion Sort é eficiente para vetores quase ordenados.
- 4. () O Quicksort sempre executa em $O(n \log n)$, independentemente da escolha do pivô.

3 – (valor: 5,0) Complete os espaços A–E no código abaixo:

```
void insertionSort(int v[], int n) {  
    for (int i = 1; i < n; i++) {  
        int chave = v[___A___];  
        int j = ___B___;  
        while (j >= 0 && v[j] ___C___ chave) {  
            v[j + 1] = v[j];  
            j = ___D___;  
        }  
        v[j + 1] = ___E___;  
    }  
}
```

4 – (valor: 4,0) Explique, em no máximo 8 linhas, como funciona o processo de intercalação (merge) no Merge Sort. Inclua na sua explicação:

- Como os subvetores são utilizados;
- Como o vetor final é reconstruído;

- Por que a intercalação garante que o vetor final esteja ordenado.

5 – (valor: 2,0) Considere uma tabela hash com **m** posições, função **$h(k) = k \bmod m$** e tratamento de colisão por encadeamento. Qual das afirmações abaixo é correta?

- A) O pior caso de busca é sempre $O(1)$.
- B) Não é possível inserir elementos que colidem.
- C) No pior caso, todos os elementos podem ficar em uma única lista.
- D) O uso de encadeamento exige que **m** seja um número primo.

6 – (valor: 4,0) Classifique como V (Verdadeiro) ou F (Falso):

- 1. () No método de endereçamento aberto nunca ocorre colisão.
- 2. () Uma boa função hash tende a distribuir chaves de forma uniforme.
- 3. () A sondagem linear pode gerar clusters (agrupamentos de chaves).
- 4. () Tabelas hash não podem armazenar valores repetidos.

7 – (valor: 4,0) Complete os espaços A–D no código a seguir:

```
void BFS(int ini, Grafo *g) {
    int fila[MAX], IF = 0, FF = 0;
    int visitado[g->n];

    for (int i = 0; i < g->n; i++)
        visitado[i] = __A__;

    visitado[ini] = 1;
    fila[FF++] = __B__;

    while (IF < FF) {
        int v = fila[IF++];
        for (int i = 0; i < g->grau[v]; i++) {
            int u = g->adj[v][i];
            if (__C__) {
                visitado[u] = 1;
                fila[FF++] = __D__;
            }
        }
    }
}
```

8 – (valor: 4,0) Descreva como funciona a busca em profundidade (DFS) em grafos, destacando:

- Como o algoritmo avança entre os vértices;
- Quando e como ocorre o backtracking;

- Limite: 10 linhas.

A) Possui pelo menos uma aresta.

C) Existe um caminho entre qualquer par de vértices.

D) Não possui ciclos.

```
int partition(int v[], int ini, int fim) {
```

```
int i = ini - 1;
```

```

if (v[j] ____ A ____ pivo) {

```

___B___; // troca v[i] com v[j]

$$\}$$
$$\}$$

```
____C____; // troca v[i+1] com v[fim]
```

```
return i + 1;
```

$$\}$$

DICA: o método da inserção para listas pode ser implementado de uma forma recursiva. A seguir descrevem-se os casos base e recursivo, respectivamente.

- [illegible]